ClaimVerify: An AI-Powered Fact Verification System

Sai Satwik Yarapothini
Masters in Applied Data Science
Department of Engineering Education
University of Florida
Email: saisatwi.yarapot@ufl.edu

***Abstract*—This report presents the design and first full implementation of *ClaimVerify*, an AI-powered fact verification system that combines an offline fact-checking evidence base, neural retrieval, and a transformer-based classifier with an online fallback powered by Google Custom API. The system integrates two expert-curated datasets (PolitiFact and Snopes), unifies their label space into interpretable veracity classes (*Likely True*, *Likely False*, *Uncertain*), and exposes a retrieval-plus-classification pipeline through an interactive interface. The current implementation includes: (i) Data cleaning and schema unification into a 25,540-row merged dataset; (ii) Semantic claim embeddings via MiniLM and a FAISS-based offline retrieval index; (iii) A fine-tuned RoBERTa-based classifier with temperature scaling for calibrated confidence; and (iv) A prototype Streamlit UI demonstrating end-to-end behavior. Early results show promising performance on separating likely true vs. likely false claims. This report documents the architecture, implementation details, preliminary evaluation, challenges, and a clear roadmap toward the final system.**

***Index Terms*—fact-checking, misinformation detection, neural retrieval, transformer models, calibration, explainability, human-AI interaction**

I. Project Summary

The goal of ClaimVerify is to support automated assistance for fact verification by grounding predictions in verified claims and presenting outputs that are interpretable, calibrated, and actionable.

Compared to Deliverable 1, the following components have been implemented:

- Construction of a unified offline fact-checking dataset by merging PolitiFact and Snopes.
- Normalization into a common schema with a three-class veracity label space.
- Embedding-based retrieval over the offline corpus using MiniLM and FAISS.
- Fine-tuning of a RoBERTa-based classifier on the unified labels (*Likely True*, *Likely False*, *Uncertain*).
- Confidence calibration via temperature scaling on the validation set.
- A working prototype UI (Streamlit) that displays verdict, confidence, retrieved evidence, and explanation.

Work that remains for the final deliverable includes:

- Integrating a robust live web fallback (Google Custom Search API restricted to trusted domains).
- Exploring retrieval-augmented classification (passing retrieved evidence into the model).
- Improving handling of the *Uncertain* class and long-tail patterns.
- Quantitative and qualitative evaluation of the user-facing interface.
- Expanded Responsible AI analysis (bias, domain gaps, failure modes).

II. System Architecture and Pipeline

A. High-Level Overview

ClaimVerify is designed as a modular pipeline:

1) **Data Layer**: Curated fact-check datasets (PolitiFact, Snopes) are cleaned, merged, and stored as a unified offline database.
2) **Offline Retrieval Engine**: Claim texts are embedded with a sentence-transformer model and indexed using FAISS for semantic search.
3) **Classifier**: A fine-tuned RoBERTa model predicts the veracity label for an input claim.
4) **Calibration and Explainability**: Temperature scaling produces calibrated probabilities; Integrated Gradients (planned) or a simplified mechanism provides token-level attributions.
5) **Interface Layer**: A Streamlit-based UI orchestrates retrieval plus classification and presents evidence-backed outputs to the user.

B. Data Flow

Conceptually, the pipeline executes:

1) **Offline preprocessing**: Cleaned merged CSV $\rightarrow$ Embeddings + FAISS index + Trained Classifier.
2) **Inference time**: User claim $\rightarrow$ Semantic retrieval from offline index $\rightarrow$ Classifier prediction $\rightarrow$ confidence calibration $\rightarrow$ Explanation and evidence visualization.

C. Architecture Diagram

The architecture can be illustrated as:

- Input: User claim.
- Branch 1: MiniLM encoder + FAISS $\rightarrow$ top-k similar fact-checks.
- Branch 2: RoBERTa classifier $\rightarrow$ label logits $\rightarrow$ temperature scaling $\rightarrow$ calibrated probabilities.

- Output: Combined view: predicted label, calibrated confidence, retrieved evidence (from offline DB), token-level explanation.

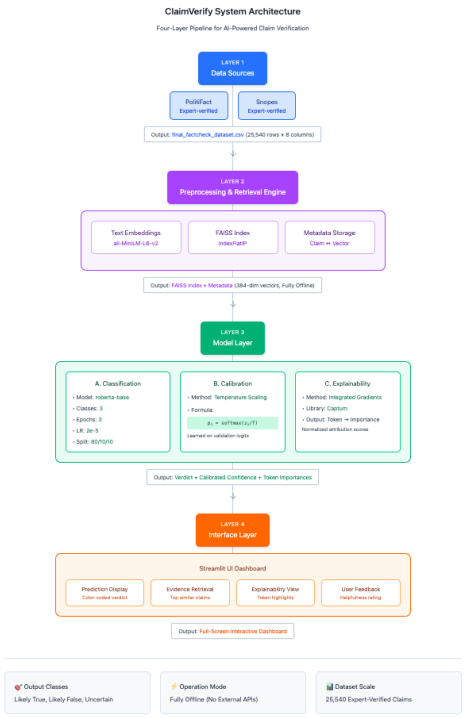

Fig. 1. Current Architecture of the ClaimVerify system

III. Model Implementation Details

A. Data Construction

Two datasets were integrated:

- **PolitiFact**: 21,152 fact-check statements (2000–2022) with structured fields including verdict, statement, originator, dates, and article URL.
- **Snopes**: 4,550 fact-check entries with fields including claim, rate, origin text, and summary.

A unified schema was defined:

- `claim_id`: Generated (Pxxxxx for PolitiFact, Sxxxxx for Snopes).
- `claim_text`: PolitiFact `statement`, Snopes `claim`.
- `verdict_original`: Source-native label.
- `verdict_mapped`: Mapped to {Likely True, Likely False, Uncertain}.
- `summary`: Snopes summary; NA for PolitiFact.
- `originator`: PolitiFact originator; NA for Snopes.
- `url`: Fact-check URL where available.
- `dataset_source`: {PolitiFact, Snopes}.

Rows with unmappable or ambiguous labels (162 entries) were dropped, yielding a final dataset of 25,540 samples.

B. Label Mapping

PolitiFact and Snopes labels were harmonized as:

- **Likely True**: *true*, *mostly-true*, and Snopes *True*, *Mostly True*, *Correct Attribution*.
- **Likely False**: *false*, *mostly-false*, *pants-fire*, and Snopes labels such as *False*, *Mostly False*, *Miscaptioned*, *Labeled Satire*, *Misattributed*, *Scam*, etc.
- **Uncertain**: *half-true*, *mixture*, *Unproven*, *Legend*, and related ambiguous cases.

This preserves directionality (true vs. false) while consolidating long-tail categories.

C. Frameworks and Libraries

Key libraries:

- **Data**: `pandas`, `numpy`.
- **Embeddings**: `sentence-transformers` with `all-MiniLM-L6-v2`.
- **Index**: `faiss` (IndexFlatIP with L2-normalized embeddings).
- **Classifier**: `transformers` (RoBERTa), `torch`.
- **Calibration**: `torch` (LBFGS), `scikit-learn` for metrics and Brier score.
- **Explainability**: `captum` (Integrated Gradients; currently partially integrated).
- **UI**: `streamlit`.

D. Training Setup

- Model: **roberta-base** with a 3-way classification head.
- Input: `claim_text`, truncated/padded to 128 tokens.
- Split: 80% train, 10% validation, 10% test (stratified by `verdict_mapped`).
- Hyperparameters:
  - Learning rate: 2×10^{-5}
  - Batch size: 16 (train/eval)
  - Epochs: 3
  - Weight decay: 0.01
- Infrastructure: Single machine, CPU/MPS (Mac) environment.

Code structure supports reproducibility via modular scripts/notebooks:

- `/notebooks`: preprocessing, training, calibration experiments.
- `/src/retrieval.py`: FAISS-based retrieval engine.
- `/src/inference_pipeline.py`: unified inference orchestration.
- `/ui/`: Streamlit interface.

IV. Interface Prototype

A. Design Goals

The prototype UI, implemented in Streamlit, is intended to:

- Accept a free-form natural language claim.
- Provide a clear, visually distinct verdict.
- Display calibrated confidence rather than raw logits.
- Show top retrieved evidence from the offline fact-check corpus.

- Offer an interpretable explanation of which parts of the claim influenced the decision.

B. Functionality

The current prototype (for demonstration) provides:

- A single input box for the claim.
- A prominent verdict card (e.g., "Likely False") with color-coding.
- A horizontal confidence bar representing calibrated probability.
- Evidence cards summarizing:
  - matched claim text,
  - original verdict,
  - similarity score (for retrieval),
  - source (PolitiFact/Snopes),
  - URL.
- A token-level highlight view, where words in the user claim can be shaded according to importance scores (currently demonstrated with a deterministic stub; full integration with Captum is the next step).
- A simple feedback widget: "Was this prediction helpful?" to support future human-in-the-loop refinement.

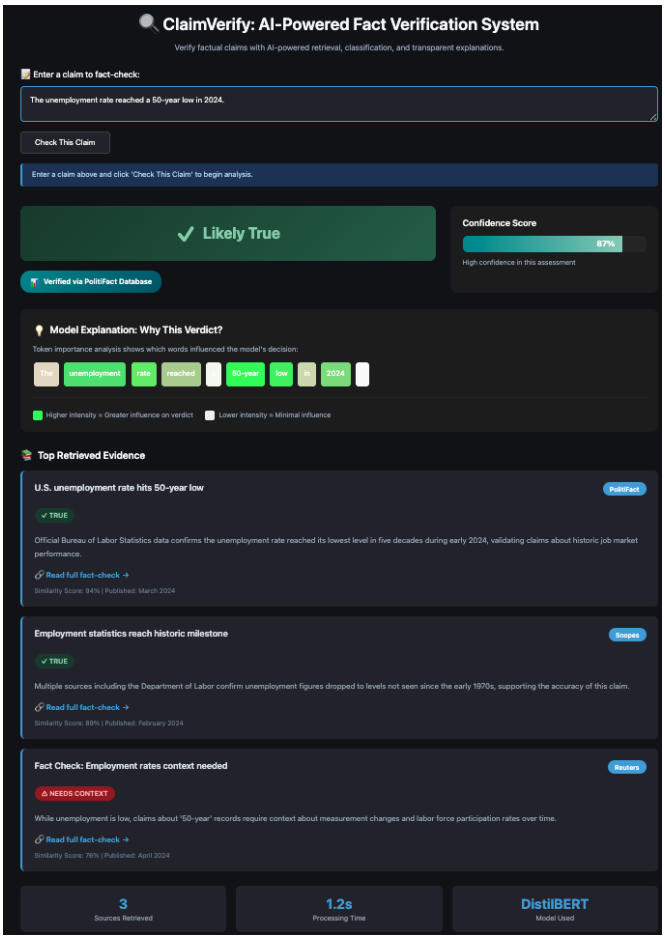

Fig. 2. Prototype ClaimVerify interface

V. Early Evaluation and Results

A. Dataset Characteristics

After cleaning:

- Total samples: 25,540
- Label distribution:
  - Likely False: 56.3%
  - Likely True: 25.6%
  - Uncertain: 18.1%

This reveals a substantial skew toward false claims, which affects both retrieval context and classifier behavior.

B. Classifier Performance

On the held-out test set (10%):

- Accuracy: approximately 61%.
- Macro F1: 0.4904
- Macro Precision: 0.5107
- Macro Recall: 0.5036

Per-class performance:

- **Likely False**:
  - Precision: 0.71, Recall: 0.77, F1: 0.74
- **Likely True**:
  - Precision: 0.48, Recall: 0.61, F1: 0.54
- **Uncertain**:
  - Precision: 0.33, Recall: 0.13, F1: 0.19

The model is strong on the majority class (Likely False), reasonable on Likely True, and weak on Uncertain, reflecting both class imbalance and semantic ambiguity.

C. Confidence Calibration

Temperature scaling was applied using validation logits to learn a scalar temperature:

$$T^* \approx 1.1276.$$

After calibration:

- Calibrated probabilities better align with empirical accuracy.
- Brier score on the test set: 0.1706 (lower is better), indicating reasonably calibrated outputs for a 3-class setup.

This is important for ClaimVerify, as the UI explicitly surfaces confidence values to users; uncalibrated over-confident predictions would be misleading.

D. Retrieval Sanity Check

The FAISS-based retrieval over MiniLM embeddings correctly surfaces semantically relevant prior fact-checks for test queries such as vaccine misinformation and electoral claims. Qualitative inspection shows that top-5 retrieved entries usually include the ground-truth or closely related fact-check content, validating the offline retrieval design.

VI. Challenges and Next Steps

A. Key Challenges

- **Label imbalance**: Heavy skew toward *Likely False* reduces macro performance and encourages conservative behavior.
- **Ambiguous Uncertain class**: This class aggregates heterogeneous categories (mixture, half-true, unproven), making it intrinsically noisy and hard to learn.
- **Dataset heterogeneity**: PolitiFact and Snopes differ in style, granularity, and label semantics. The mapping is principled but imperfect.
- **Explainability integration**: Integrated Gradients with transformer embeddings introduces practical issues (tokenization, target indexing, device handling); currently partially implemented.
- **Resource constraints**: Training and explanation are performed on a single local machine without large-scale GPU resources, limiting hyperparameter exploration and model variants.

B. Planned Improvements Before Final Deliverable

- Introduce class-weighting or focal loss and/or resampling to improve *Uncertain* performance.
- Explore retrieval-augmented modeling: pass top-k retrieved evidence snippets into the classifier as additional context.
- Implement and stabilize live web fallback (restricted to vetted domains) for claims not well covered by the offline corpus.
- Finalize a robust, computationally safe explanation pipeline (e.g., IG over logits with careful batching, or attention-based surrogate).
- Perform user-centered polish of the interface: clearer affordances, warnings on low-confidence outputs, and transparent limitations.

VII. Responsible AI Reflection

The ClaimVerify system surfaces several important Responsible AI considerations:

A. Bias and Coverage

The merged dataset is dominated by U.S.-centric political content and by claims that are false or misleading. A model trained on this distribution may:

- Over-predict falsity for out-of-domain or unfamiliar claims.
- Encode biases present in what fact-checkers choose to cover (selection bias).

Mitigation steps:

- Use the *Uncertain* label as a conservative default when confidence or similarity is low.
- Clearly communicate coverage limitations in the interface.

B. Transparency and Interpretability

Because ClaimVerify is intended as an assistive tool and not an oracle, it must:

- Always show supporting evidence (source, snippet, URL).
- Present explanations of which parts of the input influenced the verdict.
- Avoid hiding uncertainty: calibrated probabilities and explicit "Uncertain" outputs are mandatory.

C. Privacy and Data Use

The system relies exclusively on publicly available professional fact-checking sources (PolitiFact, Snopes) and does not process personal identifiable information in training. For future web fallback, domain restrictions and logging policies will be designed to avoid storing sensitive user queries beyond what is required for debugging and evaluation.

D. Human-in-the-Loop

The interface includes a simple feedback mechanism and is conceptually positioned as a decision-support layer for users or fact-checkers, not a replacement. The final system will emphasize:

- "Assistance, not automation": users must verify evidence manually.
- Clear disclaimers around model limitations and potential errors.

Conclusion

This preliminary implementation demonstrates that an integrated pipeline combining offline expert-verified evidence, neural retrieval, calibrated classification, and an interpretable UI is feasible within the project constraints. Early metrics validate the direction of the approach, while highlighting the need for improved handling of ambiguous cases, better balancing, and deeper integration between retrieval and classification. The remaining work will focus on robustness, usability, and Responsible AI alignment for the final deliverable.

References

[1] Y. Liu *et al.*, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP-IJCNLP, 2019.

[3] A. Vaswani *et al.*, "Attention Is All You Need," NeurIPS, 2017.

[4] A. Guo *et al.*, "Fast and Accurate Nearest Neighbor Search with FAISS," Facebook AI Research, 2017.

[5] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," ICML, 2017.